



University of Copenhagen
MSc. Physics

Continuous Time Finance 2

Hand-In 2

Author:

Benjamin Tariq Siddique

Contact: vhb925@alumni.ku.dk
ben123sid@gmail.com

1 – American options: The finite case

1.a

We aim to prove proposition 2.2.1. from Lamberton and Lapeyre [2]:

Proposition 2.2.1. The random variable defined by

$$\nu_0 = \inf\{n \geq 0 | U_n = Z_n\}$$

is a stopping time and the stopped sequence $(U_{n \wedge \nu_0})_{0 \leq n \leq N}$ is a martingale.

Where $(Z_n)_{0 \leq n \leq N}$ is an adapted sequence and $(U_n)_{0 \leq n \leq N}$ is its Snell envelope of the form:

$$U_n = \begin{cases} \max(Z_n, \mathbb{E}[U_{n+1} | \mathcal{F}_n]), & n = 0, \dots, N-1. \\ Z_N, & n = N. \end{cases}$$

The first part is easy to prove. First consider that a stopping time is, by definition, a random variable ν that satisfies $\{\nu = n\} \in \mathcal{F}_n$. If $\nu_0 = 0$, then the only possibility is $U_0 = Z_0$, which is obviously part of $\mathcal{F}_0$:

$$\{\nu_0 = 0\} = \{U_0 = Z_0\} \in \mathcal{F}_0.$$

And since we know that the two sequences will eventually meet, $U_N = Z_N$, we can say that for any $k \in \{1, 2, \dots, N\}$ we have:

$$\{\nu_0 = k\} = \{U_0 > Z_0\} \cap \dots \cap \{U_{k-1} > Z_{k-1}\} \cap \{U_k = Z_k\} \in \mathcal{F}_k.$$

The intuitive understanding of the above statement is that the stopping time ν_0 is the value of n at which the two sequences U_n and V_n converge, which is guaranteed to happen when $n = N$, but may well happen before, notably at $n = k$. To prove the second part of the proposition we first consider the definition of a stopped sequence U_n^ν with stopping time ν :

$$U_n^\nu = U_{n \wedge \nu} = \begin{cases} U_\nu, & \text{if } \nu \leq n. \\ U_n, & \text{if } \nu > n. \end{cases}$$

We can write the stopped sequence with stopping time ν_0 as the sum of U_0 and every step in the sequence until the stopping time is reached:

$$U_n^{\nu_0} = U_0 + \sum_{j=1}^n \mathbf{1}_{\{j \leq \nu_0\}} (U_j - U_{j-1})$$

Where $\mathbf{1}$ is the indicator function. Now using the above formula to calculate $U_{n+1}^{\nu_0} - U_n^{\nu_0}$ for $n \in \{0, 1, \dots, N-1\}$, we see that only the final term of the first sum remains:

$$U_{n+1}^{\nu_0} - U_n^{\nu_0} = U_0 + \sum_{i=1}^{n+1} \mathbf{1}_{\{n+1 \leq \nu_0\}} (U_i - U_{i-1}) - U_0 - \sum_{j=1}^n \mathbf{1}_{\{j \leq \nu_0\}} (U_j - U_{j-1}) = \mathbf{1}_{\{n+1 \leq \nu_0\}} (U_{n+1} - U_n)$$

This expression is only non-zero when $n+1 \leq \nu_0$, meaning that $U_{n+1}^{\nu_0} - U_n^{\nu_0} \neq 0 \rightarrow \forall n : U_n > Z_n$ (remember that the upper bound of n -values is $N-1$, so the condition $U_N = Z_N$ is not violated).

Now since definitionally $U_n = \max(Z_n, \mathbb{E}[U_{n+1}|\mathcal{F}_n])$, the only possibility is $U_n = \mathbb{E}[U_{n+1}|\mathcal{F}_n]$ and the above expression becomes

$$U_{n+1}^{\nu_0} - U_n^{\nu_0} = \mathbf{1}_{\{n+1 \leq \nu_0\}}(U_{n+1} - \mathbb{E}[U_{n+1}|\mathcal{F}_n]).$$

Taking the conditional expectation on both sides of the equation yields

$$\mathbb{E}[U_{n+1}^{\nu_0} - U_n^{\nu_0}|\mathcal{F}_n] = \mathbb{E}[\mathbf{1}_{\{n+1 \leq \nu_0\}}(U_{n+1} - \mathbb{E}[U_{n+1}|\mathcal{F}_n])|\mathcal{F}_n],$$

which can be further simplified by recognizing that one can move the indicator function outside of the expectation, since $\{n+1 \leq \nu_0\} \in \mathcal{F}_n$ (This is easily verified by considering the complementary case $\{\nu_0 \leq n\}$ which is clearly part of $\mathcal{F}_n$, and noting that $\mathcal{F}_n$, being a σ -algebra must be closed under complementation). Finally we arrive at the result

$$\begin{aligned} \mathbb{E}[U_{n+1}^{\nu_0} - U_n^{\nu_0}|\mathcal{F}_n] &= \mathbf{1}_{\{n+1 \leq \nu_0\}} \mathbb{E}[(U_{n+1} - \mathbb{E}[U_{n+1}|\mathcal{F}_n])|\mathcal{F}_n], \\ &= \mathbf{1}_{\{n+1 \leq \nu_0\}} (\mathbb{E}[U_{n+1}|\mathcal{F}_n] - \mathbb{E}[\mathbb{E}[U_{n+1}|\mathcal{F}_n]|\mathcal{F}_n]), \\ &= \mathbf{1}_{\{n+1 \leq \nu_0\}} (\mathbb{E}[U_{n+1}|\mathcal{F}_n] - \mathbb{E}[U_{n+1}|\mathcal{F}_n]), \\ &= 0, \end{aligned}$$

which proves that $U^{\nu_0} = (U_{n \wedge \nu_0})_{0 \leq n \leq N}$ is a martingale.

1.b

We further aim to prove proposition 2.3.1. from Lamberton and Lapeyre [2]:

Proposition 2.3.1. Every supermartingale $(U_n)_{0 \leq n \leq N}$ has the unique following decomposition:

$$U_n = M_n - A_n$$

where (M_n) is a martingale and (A_n) is a non-decreasing, predictable process, null at 0.

Suppose first that this decomposition exists. Start by considering the expression

$$U_{n+1} - U_n = M_{n+1} - M_n - (A_{n+1} - A_n).$$

After taking the expectation conditioned on $\mathcal{F}_n$ and invoking the martingale condition $\mathbb{E}[M_{n+1} - M_n] = 0$, we find

$$\mathbb{E}[U_{n+1}|\mathcal{F}_n] - U_n = -(A_{n+1} - A_n),$$

$$U_n - \mathbb{E}[U_{n+1}|\mathcal{F}_n] = A_{n+1} - A_n.$$

Note that it is now obvious that A_n is non-decreasing, due to U_n being a supermartingale, obeying $U_n - \mathbb{E}[U_{n+1}|\mathcal{F}_n] \geq 0$. Substituting into the previous equation we get

$$U_{n+1} - U_n = M_{n+1} - M_n + \mathbb{E}[U_{n+1}|\mathcal{F}_n] - U_n,$$

$$U_{n+1} - \mathbb{E}[U_{n+1}|\mathcal{F}_n] = M_{n+1} - M_n.$$

The proposition states trivially that for $n = 0$ we have $M_0 = U_0$ and $A_0 = 0$, which means that the decomposition has the following form:

$$A_n = \sum_{i=1}^n (U_{i-1} - \mathbb{E}[U_i | \mathcal{F}_{i-1}])$$

$$M_n = U_0 + \sum_{i=1}^n (U_i - \mathbb{E}[U_i | \mathcal{F}_{i-1}])$$

To prove uniqueness consider a second decomposition $X_n = M'_n - A'_n$ [3]. Let $Y_n = M_n - M'_n$ and note that this also implies $Y_n = A_n - A'_n$. Since Y_n is simultaneously a superposition of two martingales and a superposition of two predictable processes, it must be a predictable martingale, i.e. it has the properties

$$\mathbb{E}[Y_n | \mathcal{F}_{n-1}] = Y_{n-1}, \quad \mathbb{E}[Y_n | \mathcal{F}_{n-1}] = Y_n.$$

Since by construction $Y_0 = A_0 - A'_0 = 0$ we can, through iteration, conclude that $\forall n \in \{0, 1, \dots, N\} : Y_n = M_n - M'_n = A_n - A'_n = 0$, and therefore the solution is unique and the proposition has been proven.

1.c

In Poulsen [4] it is argued that replicating an american option is possible by defining the Snell envelope $Z_t = \max(g(S_t), E_t^Q[Z_{t+1}])$, where $g(S_t)$ is the intrinsic value of the option ($g(x) = (x - K)^+$ for a call option). Since Z_t is obviously a supermartingale, by proposition 2.3.1 [2] it has a Doob decomposition $Z_t = M_t - A_t$. Furthermore denoting the stopping time $\tau^* = \min(t | Z_t = g(S_t))$ makes the stopped sequence $Z_t^{\tau^*}$ a martingale (since $Z_t = \mathbb{E}_t^Q[Z_{t+1}]$ up until $t = \tau^*$), meaning $\forall t : A_t^{\tau^*} = 0$ and $Z_t^{\tau^*}$ can be perfectly replicated by a self-financing trading strategy.

Let π_t^{AMR} denote the price of the american option. The cases $\pi_t^{AMR} < Z_t$ and $\pi_t^{AMR} > Z_t$ are both covered in [4] and we now want to show that for $Z_t = \pi_t^{AMR}$ no arbitrage is possible.

3. $Z_t = \pi_t^{AMR}$. If we buy the option, then its maximum expected value is bounded by Z_t since, by Corollary 2.2.2 [2], $Z_t = \sup_{\tau^* \in \mathcal{T}_{t,T}} \mathbb{E}_t^Q[g(S_{\tau^*}) | \mathcal{F}_t]$. Thus no arbitrage is possible in this way. On the other hand if we sell the option and buy the self-financing trading strategy that replicates the martingale part of Z_t our portfolio will have the value $M_t = Z_t + A_t$, which is subject to change depending on when the buyer exercises the option. If the buyer exercises optimally at τ^* , we make zero profit since, as stated previously, $\forall t : A_t^{\tau^*} = 0$ and $M_{\tau^*} = Z_{\tau^*} = g(S_{\tau^*})$. If the buyer does not exercise optimally, then at some point $A_t > 0$ and we actually make a profit because $M_t = Z_t + A_t > g(S_t)$. So if both buyer and seller act optimally, no arbitrage is possible when $Z_t = \pi_t^{AMR}$.

1.d

We extend the argumentation in points 1.-2. in Poulsen [4] for the cases where the interest rate is non-zero. Thus it is now possible to invest in a risk-free asset, and any payoff must be discounted

accordingly. The discounted intrinsic value of an american option now becomes $\tilde{g}(S_t) = \frac{g(S_t)}{(1+r)^t}$ and its Snell envelope is $\tilde{Z} = \max(\tilde{g}(S_t), \mathbb{E}_t^Q[\tilde{Z}_{t+1}])$. Note that

$$\begin{aligned}\tilde{Z}_t &= \frac{1}{(1+r)^t} Z(t) = \max \left(\frac{1}{(1+r)^t} g(S_t), \mathbb{E}_t^Q \left[\frac{1}{(1+r)^{t+1}} Z_{t+1} \right] \right), \\ Z(t) &= \max \left(g(S_t), \mathbb{E}_t^Q \left[\frac{1}{(1+r)} Z_{t+1} \right] \right).\end{aligned}$$

Now consider the two cases:

1. $\pi_t^{AMR} < Z_t$. We buy the american option, financing it with the money obtained from selling the trading strategy that replicates the stopped discounted process $\tilde{Z}_t^{\tau^*}$. The current market value (cost) of this replicating portfolio is Z_t , meaning we make an immediate arbitrage profit of $Z_t - \pi_t^{AMR}$. When we then exercise our option at the optimal stopping time τ^* , the intrinsic value $g(S_{\tau^*})$ perfectly matches whatever value $\tilde{Z}_{\tau^*}$ has taken on, allowing us to settle our remaining investments.

2. $\pi_t^{AMR} > Z_t$. We sell the american option and use the funds to buy a trading strategy that replicates the martingale part of $\tilde{Z}_t$ (which is priced at Z_t). In other words our portfolio holds the value $M_t = \tilde{Z}_t + A_t$. Since $A_t \geq 0$ we know that $M_t \geq \tilde{Z}_t$ at any given time. It is clear that we have thus made an arbitrage profit of (at least) $\pi_t^{AMR} - Z_t$.

The case of a non-zero interest rate is of considerable financial importance because it reflects the reality of real-world markets, where one should account for the time-value of money. If we simply ignored this fact it would, for example, not be possible to explain why it is never optimal to exercise an american call option early.

1.e

We now want to show, by constructing a counterexample, that replicating a stopped Snell envelope (which is a Q -martingale), does not guarantee an arbitrage profit, when $\pi_t^{AMR} > Z_t$ and therefore the doob decomposition is a necessary part of the proof in Poulsen [4]. To do so we consider a two-period binomial model of an american put option with $K = 10$, $S_0 = 10$, $u = 5$, $d = \frac{1}{5}$ and $r = 0$. I have drawn a below scheme to make the visualization easier:

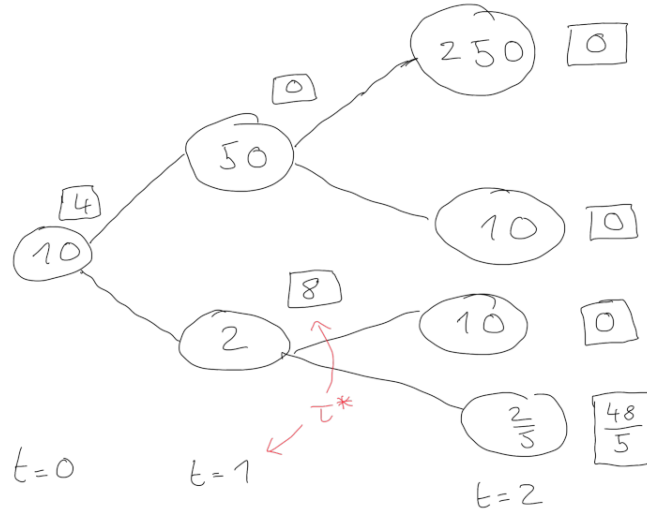


Figure 1: Two period binomial model with strike price $K = 10$.

The circles contain the price of the underlying stock, while the boxes denote the value of the Snell envelope $Z_t = \max(K - S(t), \mathbb{E}_t^Q[Z_{t+1}])$. The optimal stopping time, if the stock path experiences an initial downturn, is clearly $\tau^* = 1$. Let us now consider what would happen if we sold the put option at the price $\pi_t^{AMR} = 5$, and used that to buy the trading strategy that replicates $Z_t^{\tau^*}$ (which is valued at 4 when $t = 0$). Now if the buyer of the put chooses to exercise optimally at τ^* , they receive a payoff of 8, which we have perfectly hedged with our replicating portfolio, allowing us to pay the buyer in full. But if the buyer does not exercise optimally, and the stock goes down at $t = 2$, then our arbitrage fails and we lose money ($8 + 1 < \frac{48}{5}$). Therefore we cannot just replicate $Z_t^{\tau^*}$ and the Doob decomposition is a necessary part of the proof in Poulsen [4].

2 – American Options: Longstaff and Schwartz

2.a

We aim to replicate σ_{RV} and σ_{ML} from table 1 in Poulsen [5]. Both parameters are extracted from sample stock paths from Longstaff and Schwartz [6]. The first value σ_{RV} is estimated using the standard deviation of the log-returns $t = \{1, 2, 3\} : \xi_t = \ln\left(\frac{S(t)}{S(t-1)}\right)$. This is based on the fact that the stock paths are assumed to follow a geometric brownian motion. The standard error on σ_{RV} is approximately $\frac{\sigma_{RV}}{\sqrt{2(M-1)}}$, where M is the total number of log-returns (In our case $3 \cdot 8 = 24$). Meanwhile σ_{ML} is determined using a maximum likelihood estimation, motivated by the fact that Black-Scholes assumes i.i.d. log-returns according to a normal distribution of the form $\xi \sim \mathcal{N}\left(r - \frac{1}{2}\sigma_{ML}^2, \sigma_{ML}^2\right)$. Both parameter estimations were done in python, where a log-likelihood estimator was implemented for the latter, i.e. we aimed to maximize

$$\ln(L(\xi_{\text{matrix}}, \sigma)) = -\frac{1}{2} \sum_{m=1}^M \frac{(\xi(m) - (r - \frac{1}{2}\sigma^2))^2}{\sigma^2} - M \ln(\sigma) - \frac{M}{2} \ln(2\pi),$$

w.r.t. σ . Using this formulation we can also determine the uncertainty of σ_{ML} , which is given as

$$\sigma_{ML, err} = \sqrt{-\left(\frac{\partial^2 L}{\partial^2 \sigma}\right)^{-1}_{\sigma=\sigma_{ML}}}.$$

A closed form solution exists, but for our purposes a simple a finite difference method is enough, i.e. we approximate the double differential as:

$$\left(\frac{\partial^2 L}{\partial^2 \sigma}\right)_{\sigma=\sigma_{ML}} \approx L(\xi_{\text{matrix}}, \sigma_{ML} - h) + L(\xi_{\text{matrix}}, \sigma_{ML} + h) - 2L(\xi_{\text{matrix}}, \sigma_{ML}),$$

where h is some small value ($h = 10^{-5}$ in my specific implementation). Using these methods we have succesfully replicated $\sigma_{RV} = 0.1519 \pm 0.22$ and $\sigma_{ML} = 0.1484 \pm 0.021$

2.b

We now aim to replicate the put option prices from table 2 in Poulsen [5]. These prices are determined by using a binomial tree, for varying volatilities σ , and iterating backwards along it to determine the value of the options. The european put option was the simplest case of such and is also described in detail in Björk chapter 2 [1] (Albeit he only covers the call option). The price of the american option was determined by continously comparing its expected future value to the profit from its immediate exercise. The Bermudan option could only be exercised at the end of the year, which we accounted for by discretizing each year into m steps in the binomial tree. Based on the code provided in the article I implemented my own solution in python with a binomial tree running over 3 years and $m = 252$, which yielded the following:

σ	American	Bermudan	European
0.1484	0.1085	0.0918	0.0628
0.1500	0.1092	0.0927	0.0637
0.1484	0.1099	0.0938	0.0649

Table 1: Calculated put option prices using a binomial tree.

Some of my calculated prices differ on the last decimal from those presented in Poulsen [5] and I'm not sure why. I suspect that the put prices in the paper may have been calculated using a different value for m , than the one used in the linked code. Assuming that Longstaff and Schwartz used $\sigma = 0.1500$ in their simulations, we see that our value for the price of the American put option is close, but slightly below their estimated valuation at 0.1144.

2.c

In question 1 of exercises for the reader in Poulsen [5], we are asked to determine the impact on the price of the of the LSM American put option price, with increasing degrees on our polynomial fits, which we have done below:

Degree	Price
2	0.1144
3	0.1154
4	0.1243
5	Error

Table 2: Table reflecting an increase in price of the LSM American put option, when increasing the degrees of the polynomial used to determine the continuation value at a given timestep.

When allowing for higher degree polynomials in our LS approach, the price of the option increases. This is to be expected, due to the fact that we can now much more accurately determine the continuation values for these particular paths. The payoff is of course that we are almost surely overfitting, capturing noise in the stock prices rather than any intrinsic patterns of their dynamics (It would for example severely negatively affect our expected payoff in 2e, where we attempt to extrapolate the Longstaff and Schwartz derived coefficients to new stock paths). Note that fitting a fifth-degree polynomial yields an error, which is to be expected, since we are essentially trying to estimate 6 parameters, when we only have 5 independent equations. In our implementation we used matrix operations to determine the least-squares solution, but using a built in regressor (like `lm` in R or `np.linalg.lstsq` in Python) would yield the exact same results, except it may not give an error for the fifth-degree polynomial. This is due to the fact that these solvers use other methods (such as singular value decomposition) to still determine the 'best fit' on the data.

2.d

Moving onto question 2 of Poulsen [5], we now simulate 8 different stock paths, with dynamics $dS(t) = 0.06S(t)dt + 0.15S(t)dW(t)$, for 3 terms, a total of 10000 times. The resulting distribution

looks like a gaussian, that is also slightly asymmetric, due to a truncation at 0 (because negative prices are not possible). The mean is found to be $\mu = 0.1061$ with a standard deviation of $\sigma = 0.037$.

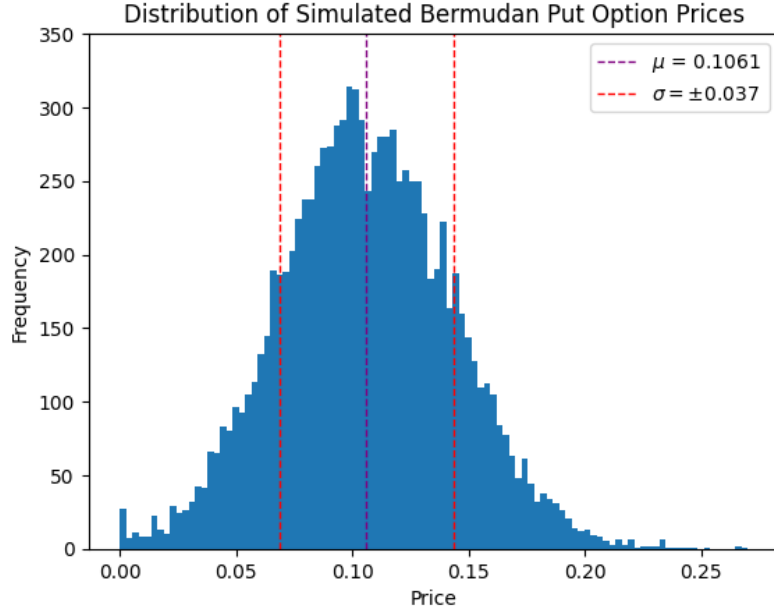


Figure 2: Resulting distribution of prices for Bermudan put options based on the LS simulation method. In total $8 \times 10,000$ stock paths were simulated.

2.e

In question 3 from Poulsen [5] we are tasked to use the specific regression parameters obtained from the example in Longstaff and Schwartz [6] and extrapolate them for many newly generated stock paths. For this problem the simulations are rather straightforward, since we can immediately calculate the continuation value for a given timestep, and can therefore also iterate chronologically i.e. no back-propagation is needed. Simulating 10^7 stock paths consistently yields an average discounted payoff of ~ 0.0900 , which is well below the true value of a bermudan put option with $\sigma = 0.15$, which we know to be ~ 0.0927 . This reflects the fact that basing our model parameters on merely 8 stock paths is not enough to capture the overall dynamics of the stock movements. We essentially have so little data, that the determined coefficients end up containing a fair bit of noise, affecting their extrapolation value.

References

- [1] T. Björk, Arbitrage Theory in Continuous time, 4th edition. Oxford University Press (2020).
- [2] D. Lamberton and B. Lapeyre, Introduction to Stochastic Calculus Applied to Finance, 2nd edition. CRC Press (2008).
- [3] Wikipedia contributors, Doob decomposition theorem, Wikipedia (2025).
URL https://en.wikipedia.org/w/index.php?title=Doob_decomposition_theorem&oldid=1303397663.
- [4] R. Poulsen, American π : Piece of Cake? WILMOTT magazine (2017).
- [5] R. Poulsen, Secrets of Longstaff and Schwartz, Aged 23 3/4: Part I, The Example. WILMOTT magazine (2025).
- [6] F. A. Longstaff and E. S. Schwartz, Valuing American Options by Simulation: A Simple Least-Squares Approach. The review of Financial Studies (2001).

Hand-in 2

March 9, 2026

1 Hand-In 2

```
[1]: import numpy as np
import matplotlib.pyplot as plt
```

1.1 2a)

```
[2]: S1 = np.array([1, 1.09, 1.08, 1.34])
S2 = np.array([1, 1.16, 1.26, 1.54])
S3 = np.array([1, 1.22, 1.07, 1.03])
S4 = np.array([1, 0.93, 0.97, 0.92])
S5 = np.array([1, 1.11, 1.56, 1.52])
S6 = np.array([1, 0.76, 0.77, 0.90])
S7 = np.array([1, 0.92, 0.84, 1.01])
S8 = np.array([1, 0.88, 1.22, 1.34])

stock_prices = np.vstack((S1, S2, S3, S4, S5, S6, S7, S8))

#Calculate sigma_RV
log_returns = np.log(stock_prices[:, 1:] / stock_prices[:, :-1])
sigma_RV = np.std(log_returns, ddof = 1)
print('sigma_RV = ', round(sigma_RV, 4), '+/-', round(sigma_RV/np.
    ↳sqrt(2*(log_returns.size - 1)), 3))

#Calculate sigma_ML
r = 0.06
max_likelihood = -np.inf
sigma_vals = np.linspace(0.1, 0.2, 1000)
for sigma in sigma_vals:
    log_likelihood = -0.5 * np.sum(((log_returns - (r - 0.5 * sigma**2)) /
    ↳sigma)**2) - log_returns.size * np.log(sigma) - 0.5 * log_returns.size * np.
    ↳log(2 * np.pi)
    if log_likelihood > max_likelihood:
        max_likelihood = log_likelihood
        sigma_ML = sigma

#Calculate the standard error of sigma_ML
```

```

h = 10**(-5)
log_likelihood_plus = -0.5 * np.sum(((log_returns - (r - 0.5 * (sigma ML +
↪h)**2)) / (sigma ML + h)**2) - log_returns.size * np.log(sigma ML + h) - 0.
↪5 * log_returns.size * np.log(2 * np.pi)
log_likelihood_minus = -0.5 * np.sum(((log_returns - (r - 0.5 * (sigma ML -
↪h)**2)) / (sigma ML - h)**2) - log_returns.size * np.log(sigma ML - h) - 0.
↪5 * log_returns.size * np.log(2 * np.pi)
second_derivative = (log_likelihood_plus - 2 * max_likelihood +
↪log_likelihood_minus) / h**2
sigma ML_error = np.sqrt(- 1 / second_derivative)

print('sigma ML = ', round(sigma ML, 4), '+/-', round(sigma ML_error, 3))

```

sigma_RV = 0.1519 +/- 0.022

sigma ML = 0.1484 +/- 0.021

1.2 2b)

```

[3]: def binomial_puts(S0, T, strike, r, sigma, n, m, option_type = 'European'):
    dt = T/n                                #time step
    u = np.exp(sigma * np.sqrt(dt))          #up movement in stock
    ↪price
    d = np.exp(-sigma * np.sqrt(dt))         #down movement in stock
    ↪price
    R = np.exp(r * dt)                       #risk-free growth factor
    q = (R - d) / (u - d)                   #risk-neutral
    ↪probability of up movement

    St_maturity = S0 * u**np.arange(0, n+1) * d**np.arange(n, -1, -1)  #stock
    ↪prices at maturity
    price = np.maximum(0, strike - St_maturity)
    ↪#possible option values at maturity

    for i in range(n, 1, -1):
        St = S0 * u**np.arange(0, i) * d**np.arange(i-1, -1, -1)
        ↪#stock prices at time step i - 1, where i is descending from n to 2
        exercise_value = np.maximum(0, strike - St)
        ↪#exercise value at time step i - 1

        if option_type == 'European':
            ↪#european option cannot be exercised before maturity
            exercise_value = 0

        if option_type == 'American':
            ↪#american option can be exercised at any time
            exercise_value = np.maximum(0, strike - St)

```

```

        if option_type == 'Bermudan':
            ↪ #bermudan option can only be exercised at the end of the year
            exercise_value = exercise_value * (i % m == 0)

            continuation_value = (q * price[1 : i + 1] + (1 - q) * price[0 : i]) / R
            ↪R          #expected value of holding the option, discounted back to time step
            ↪i - 1
            price = np.maximum(continuation_value, exercise_value)
            ↪ #check if expected value of holding the option is less than the exercise
            ↪value. If so, exercise the option

        return (q * price[1] + (1 - q) * price[0]) / R
        ↪ #price at time 0, discounted back to present value

sigmas = [sigma_ML, 0.1500, sigma_RV]
m_year = 252                                #possible exercise dates per year
n_total = 3 * m_year                        #total time steps for binomial tree

for s in sigmas:
    price_european = binomial_puts(1, 3, 1.1, r, s, n_total, m_year,
    ↪option_type = 'European')
    price_american = binomial_puts(1, 3, 1.1, r, s, n_total, m_year,
    ↪option_type = 'American')
    price_bermudan = binomial_puts(1, 3, 1.1, r, s, n_total, m_year,
    ↪option_type = 'Bermudan')
    print(f'For sigma = ', round(s, 4), ': American put price = ',
    ↪round(price_american, 6), 'Bermudan put price = ', round(price_bermudan, 6),
    ↪'European put price = ', round(price_european, 6))

```

```

For sigma = 0.1484 : American put price = 0.108546 Bermudan put price =
0.091827 European put price = 0.062752
For sigma = 0.15 : American put price = 0.109175 Bermudan put price = 0.092719
European put price = 0.063706
For sigma = 0.1519 : American put price = 0.109938 Bermudan put price =
0.093794 European put price = 0.064869

```

1.3 2c)

```

[4]: def solve_lsm(S, strike, r, degree = 2):
    paths, steps = S.shape
    dt = 1
    cash_flows = np.maximum(strike - S[:, -1], 0)
    exercise_times = np.full(paths, steps - 1)

    for t in range(steps - 2, 0, -1):
        itm_mask = S[:, t] < strike #in the money mask

```

```

        if np.sum(itm_mask) > 0:
            X = S[itm_mask, t]
            Y = cash_flows[itm_mask] * np.exp(-r * (exercise_times[itm_mask] -
↪t) * dt)
            A = np.vander(X, degree + 1, increasing = True)
↪        #create matrix with 1, X, X^2 etc.
            coeffs = np.linalg.inv(A.T @ A) @ A.T @ Y
↪        #least squares

            continuation_value = A @ coeffs
            intrinsic_value = strike - X

            exercise_mask = intrinsic_value > continuation_value
            exercise_indices = np.where(itm_mask)[0][exercise_mask]

            cash_flows[exercise_indices] = intrinsic_value[exercise_mask]
            exercise_times[exercise_indices] = t

        #take average of discounted cash flows for final price
        final_discounted_payoffs = cash_flows * np.exp(-r * exercise_times * dt)
        return np.mean(final_discounted_payoffs)

d_vals = [2, 3, 4, 5]
for d in d_vals:
    try:
        price = solve_lsm(stock_prices, strike = 1.1, r = 0.06, degree = d)
        print('Degree %s price: %s' % (d, round(price, 4)))
    except np.linalg.LinAlgError:
        print('Degree %s: Error, could not compute price.' % d)

```

```

Degree 2 price: 0.1144
Degree 3 price: 0.1154
Degree 4 price: 0.1243
Degree 5: Error, could not compute price.

```

1.4 2d)

```

[5]: S0 = 1.0
     K = 1.1
     r = 0.06
     sigma = 0.15
     T = 3
     steps = 3
     paths_per_exp = 8
     N_sim = 10000
     dt = 1.0

```

```

def solve_lsm_8paths(S, strike, r_rate):
    paths, total_steps = S.shape
    P = np.zeros((paths, total_steps))          #cash flow matrix
    P[:, -1] = np.maximum(strike - S[:, -1], 0)

    for t in range(total_steps - 2, 0, -1):
        itm_mask = S[:, t] < strike
        if np.sum(itm_mask) < 3:                  #we cant fit a 2nd degree
            ↪polynomial with only 2 points or less
            continue

        X = S[itm_mask, t]
        Y = np.zeros(np.sum(itm_mask))
        A = np.vander(X, 3, increasing = True)

        #calculate discounted future cash flows for in the money paths
        for i, path_idx in enumerate(np.where(itm_mask)[0]):
            for future_t in range(t + 1, total_steps):
                if P[path_idx, future_t] > 0: #if there is a cash flow in the
                    ↪future, discount it back to time t and use it as Y value for regression
                    Y[i] = P[path_idx, future_t] * np.exp(-r_rate * (future_t -
                    ↪t))

                break

        coeffs = np.linalg.inv(A.T @ A) @ A.T @ Y
        continuation_value = A @ coeffs
        intrinsic_value = strike - X

        exercise_mask = intrinsic_value > continuation_value
        exercise_indices = np.where(itm_mask)[0][exercise_mask]

        P[exercise_indices, t] = intrinsic_value[exercise_mask]
        P[exercise_indices, t+1:] = 0              #zero out
        ↪future cash flows

        payoffs = np.zeros(paths)
        for i in range(paths):
            for t in range(1, total_steps):
                if P[i, t] > 0:
                    payoffs[i] = P[i, t] * np.exp(-r_rate * t)      #discount cash
                    ↪flow back to present value
                break
        return np.mean(payoffs)

#Simulation loop
results = []

```

```

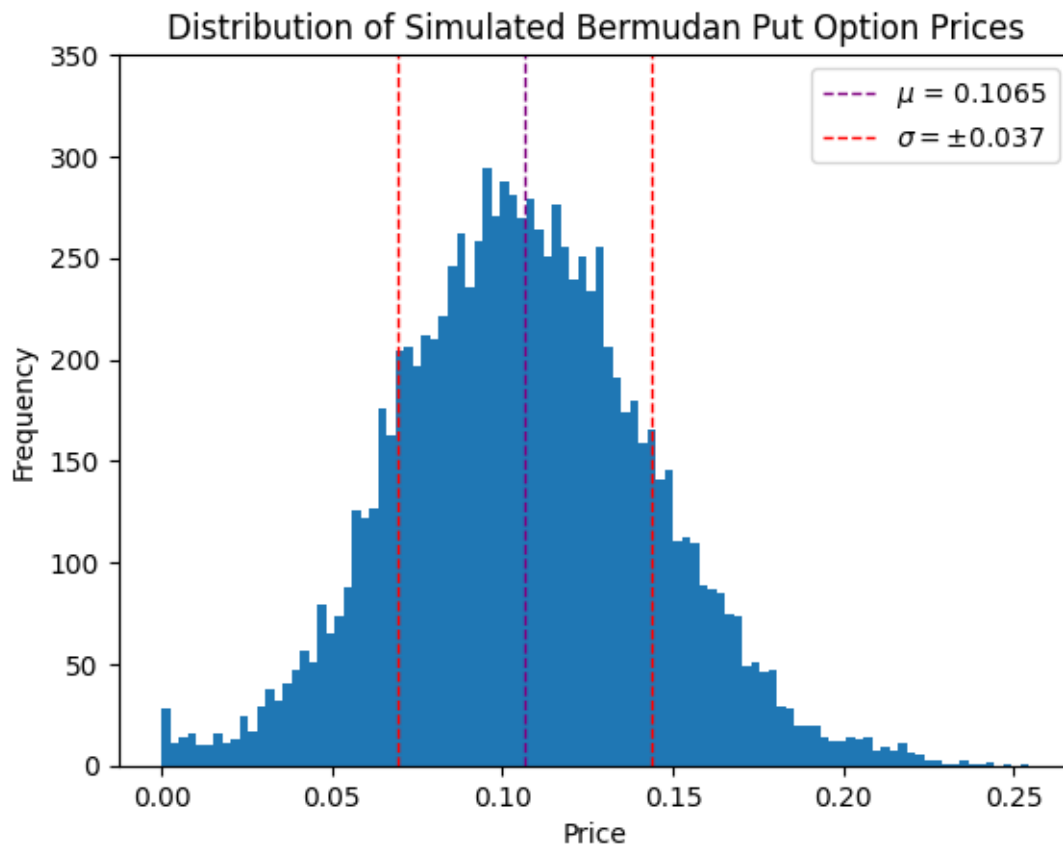
for _ in range(N_sim):
    S = np.zeros((paths_per_exp, steps + 1))
    S[:, 0] = S0
    for t in range(1, steps + 1):
        Z = np.random.standard_normal(paths_per_exp)
        S[:, t] = S[:, t-1] * np.exp((r - 0.5 * sigma**2) * dt + sigma * np.
↳sqrt(dt) * Z)

    results.append(solve_lsm_8paths(S, K, r))
results = np.array(results)

#Plot
print('Mean option price: ', round(np.mean(results), 4))
print('Standard deviation: ', round(np.std(results, ddof=1), 4))
plt.hist(results, bins = 100)
plt.vlines(np.mean(results), 0, 1100, color='purple', linestyle='dashed',
↳linewidth=1, label='$\mu$ = %s' % round(np.mean(results), 4))
plt.vlines(np.mean(results) + np.std(results, ddof=1), 0, 1100, color='red',
↳linestyle='dashed', linewidth=1, label='$\sigma = \pm %s$' % round(np.
↳std(results, ddof=1), 3))
plt.vlines(np.mean(results) - np.std(results, ddof=1), 0, 1100, color='red',
↳linestyle='dashed', linewidth=1)
plt.ylim(0, 350)
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.legend()
plt.title('Distribution of Simulated Bermudan Put Option Prices')
plt.show()

```

Mean option price: 0.1065
Standard deviation: 0.0373



1.5 2e)

```
[6]: N_sim = 10**7

#Regression coefficients from Longstaff and Schwartz
B1 = np.array([2.037512, -3.335443, 1.356457])
B2 = np.array([-1.06998, 2.983411, -1.813576])

#Simulate Paths
Z = np.random.standard_normal((N_sim, 3))
S = np.zeros((N_sim, 4))
S[:, 0] = S0
for t in range(1, 4):
    S[:, t] = S[:, t-1] * np.exp((r - 0.5 * sigma**2) * dt + sigma * np.
    ↪sqrt(dt) * Z[:, t-1])

def fixed_strategy(S, K, r, B1, B2):
    payoffs = np.zeros(N_sim)
```

```

for i in range(N_sim):
    #t=1
    if S[i, 1] < K:
        intrinsic_value = K - S[i, 1]
        continuation_value = B1[0] + B1[1]*S[i, 1] + B1[2]*(S[i, 1]**2)
        if intrinsic_value > continuation_value:
            payoffs[i] = intrinsic_value * np.exp(-r * 1)
            continue

    #t=2
    if S[i, 2] < K:
        intrinsic_value = K - S[i, 2]
        continuation_value = B2[0] + B2[1]*S[i, 2] + B2[2]*(S[i, 2]**2)
        if intrinsic_value > continuation_value:
            payoffs[i] = intrinsic_value * np.exp(-r * 2)
            continue

    #t = 3
    payoffs[i] = max(K - S[i, 3], 0) * np.exp(-r * 3)

return np.mean(payoffs)

expected_payoff = fixed_strategy(S, K, r, B1, B2)
print(f"Expected payoff from fixed strategy: {expected_payoff:.5f}")

```

Expected payoff from fixed strategy: 0.09000